

Predicting complex time series with deep echo state networks

Special Collection: [Nonlinear Dynamics of Reservoir Computing: Theory, Realization and Application](#)

Afrouz Delshad  ; Elizabeth M. Cherry  



Chaos 35, 093126 (2025)

<https://doi.org/10.1063/5.0283425>



Articles You May Be Interested In

Predicting bifurcation of mechanical systems using reservoir computing: Case studies on legged locomotion and pneumatic soft actuator

Chaos (January 2026)

A machine-learning approach for long-term prediction of experimental cardiac action potential time series using an autoencoder and echo state networks

Chaos (June 2022)

Predicting critical transitions in multiscale dynamical systems using reservoir computing

Chaos (December 2020)



Chaos

**Special Topics Open
for Submissions**

[Learn More](#)

Predicting complex time series with deep echo state networks

Cite as: Chaos 35, 093126 (2025); doi: 10.1063/5.0283425

Submitted: 31 May 2025 · Accepted: 23 August 2025 ·

Published Online: 10 September 2025



View Online



Export Citation



CrossMark

Afrouz Delshad and Elizabeth M. Cherry^{a)}

AFFILIATIONS

School of Computational Science and Engineering, Georgia Institute of Technology, Atlanta, Georgia 30332, USA

Note: This paper is part of the Special Topic on Nonlinear Dynamics of Reservoir Computing: Theory, Application and Realization.

^{a)}Author to whom correspondence should be addressed: elizabeth.cherry@gatech.edu

ABSTRACT

Although many real-world time series are complex, developing methods that can learn from their behavior effectively enough to enable reliable forecasting remains challenging. Recently, several machine-learning approaches have shown promise in addressing this problem. In particular, the echo state network (ESN) architecture, a type of recurrent neural network where neurons are randomly connected and only the read-out layer is trained, has been proposed as suitable for many-step-ahead forecasting tasks. ESNs have been shown to be effective in predicting complex time series while reducing training costs, thereby improving efficiency. Deep echo state networks, which extend the architecture of ESNs by stacking multiple reservoir layers, have also been proposed as an option to learn different features of the dynamics across the layers, but to date they have received less study. In this work, we analyzed the performance of deep ESNs for several variations in network structure, including a hybrid approach that integrated a knowledge-based model. We found that for specific network structures, deep ESNs could improve prediction accuracy over baseline ESNs with the same number of neurons by up to 65% for a chaotic data set derived from the Mackey–Glass model and 14% for an experimental data set of complex zebrafish cardiac voltage recordings. Similarly, deep hybrid ESNs could reduce error compared to same-size flat hybrid ESNs by up to 59% for the Mackey–Glass data set and 11% for the experimental data set. Our results also showed that the hybrid approach was especially beneficial for the experimental data set and that deep network structures improved prediction robustness.

Published under an exclusive license by AIP Publishing. <https://doi.org/10.1063/5.0283425>

The dynamics of time series of cardiac electrical signals can be highly complex, and the ability to foresee potential irregularities in the heart's activity could aid in developing effective preventative solutions. However, accurate multi-step-ahead prediction is a difficult task that can be computationally demanding, especially for data-driven approaches. With this motivation, we extended previous work studying the forecasting ability of computationally efficient echo state networks (ESNs) for complex cardiac time series by examining a multi-layer deep ESN (DESN) method, along with a deep hybrid ESN (DHESN) that incorporated input from a knowledge-based model to help guide predictions. We evaluated different implementation strategies for these approaches using complex time series both from the widely used Mackey–Glass model and from experimental cardiac voltage recordings from zebrafish hearts. For these data sets, certain implementations of DESN and DHESN architectures that allowed features represented in the layers to be preserved

in the output consistently outperformed flat ESN and HESN approaches, respectively, when configured with the same total number of neurons. We also found that the benefits of hybrid methods were problem-dependent; while HESN and DHESN did not offer advantages for the Mackey–Glass data set, their use led to improved prediction accuracy for the cardiac data set.

I. INTRODUCTION

Accurate prediction of complex real-world time series for a meaningful period of time could offer many benefits for understanding and controlling systems. For example, predictions of cardiac voltage signals might allow early intervention to prevent the development of an alternating cardiac rhythm that frequently progresses to fibrillation,^{1,2} or could help to identify the most effective times to

apply defibrillation.³ However, forecasting complex nonlinear time series remains a challenging task.

A variety of approaches have been proposed to solve this type of problem.⁴ In particular, machine learning (ML) methods have proven effective for time-series forecasting, with applications ranging from economics and finance⁵ to rainfall prediction⁶ and healthcare.⁷ The high training costs associated with traditional neural network approaches have motivated the development of alternatives including echo state networks,^{8,9} where a recurrent neural network (RNN) called the reservoir is generated randomly and only the weights of the readout layer are trained. ESNs along with liquid state machines are known as reservoir computing, an approach that has facilitated the practical application of RNNs and has outperformed classical fully trained RNNs in many tasks.¹⁰ When available, mechanistic models can be used in conjunction with the reservoir computing approach in a hybrid forecasting scheme to produce more accurate predictions.¹¹ Some studies have also combined ESNs with other ML methods to achieve higher accuracy, such as using an autoencoder with the echo state network.¹²

Recently, baseline ESNs, clustered ESNs (CESNs) with a reservoir of sparsely connected sub-graphs, and hybrid ESNs (HESNs) incorporating knowledge-based mechanistic models have been studied to predict complex time series from both synthetic and experimental data sets. Compared to traditional ML techniques of long short-term memory (LSTM) networks and gated recurrent units (GRUs), these ESN approaches were shown to offer improvements in accuracy while reducing the computational costs by one to two orders of magnitude.¹³ However, further methodological refinements could offer further improvements in prediction accuracy, computational demands, or robustness. In particular, inspiration may be drawn from the success of deep learning, which has been established as a useful prediction tool for time-series data in various fields.^{14–18} Recently, a deep ESN model¹⁹ was introduced, thereby uniting the merits of deep learning and ESNs to support learning different temporal dynamics across the network layers.²⁰

In this work, we implement for the first time a deep ESN approach for predicting complex experimental cardiac voltage time series¹³ and also analyze deep ESN forecasting performance for a synthetic data set derived from the Mackey–Glass model²¹ of white blood cell production.²² We analyze the accuracy and robustness of time-series predictions many steps ahead for these data sets. In particular, we compare results from baseline and hybrid ESNs with deep ESNs (DESNs) and deep hybrid ESNs (DHESNs), considering variations of network connections to maximize learning in the layered structure.

In the rest of this work, we begin by explaining the methods in Sec. II, including two variations of the baseline ESN and HESN, six variations of DESNs and DHESNs, and Bayesian optimization, which was employed both for hyperparameter tuning of the machine learning models and for identifying suitable parameter sets for the knowledge-based models used for the cardiac data set. Section III describes the Mackey–Glass and experimental cardiac voltage data sets, hyperparameter selection, and error assessment methods. In Sec. IV, we present the forecasting results obtained for the two data sets for the network structures considered. Finally, Sec. V provides a summary and discussion of our findings, limitations, and future work.

II. METHODS

In this section, we describe the structures and equations used for basic, hybrid, deep, and deep hybrid echo state networks and provide a brief overview of Bayesian optimization.

A. Echo state network implementation

1. Basic echo state network (ESN)

An echo state network (ESN) is a reservoir computing approach with sparsely connected neurons. The low computational cost of the ESN stems from the fact that only the output layer is trained, while the rest of the network remains untrained. As illustrated in Fig. 1(a), the input layer transmits data to the reservoir, which consists of neurons that are randomly connected. The weights assigned to the connections between neurons are also initialized randomly. The reservoir state h_t for each neuron at time step t is computed as follows:

$$h_t = (1 - \alpha)h_{t-1} + \alpha \tanh(W_{in}x_t + Wh_{t-1}), \quad (1)$$

where x_t is the input signal, W_{in} represents the input weight matrix, W refers to the reservoir weight matrix, and α is the leaking rate associated with a linear history term. For each time point t , the output is calculated as

$$y_t = f_{out}(W_{out}h_t), \quad (2)$$

where f_{out} is the activation function used for the output layer, chosen as the unity function in this paper, and W_{out} refers to the readout weights, which are found by least-squares regression with Tikhonov regularization using a regularization parameter value of 10^{-5} .

As indicated by the blue dashed line in Fig. 1(a), we also consider a second scenario where the input is fed directly to the output layer. When the input is not fed to the output layer, Eq. (2) is used to compute the output at each time step. However, when the input is included in the output layer, a variant we refer to as ESN+, the

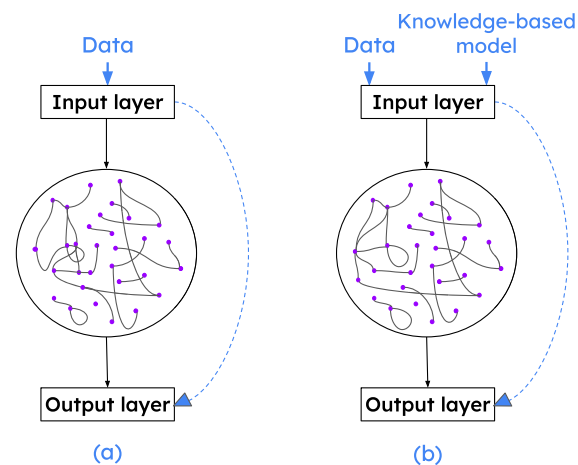


FIG. 1. (a) Basic echo state network (ESN) and (b) hybrid echo state network (HESN) architectures.

output is computed as follows:

$$y_t = f_{\text{out}}(W_{\text{out}}[x_t; h_t]). \quad (3)$$

Once the output weights at time t are calculated, the output value at time $t + 1$ is obtained by feeding the prediction results from each time step back into the network as input for the next time step, thereby allowing multi-step-ahead prediction.

2. Hybrid echo state network (HESN)

A physics-informed version of ESN, known as hybrid ESN (HESN), enables this machine-learning approach to benefit from both data-driven learning and mechanistic mathematical models that are useful but imperfect. In this study, to incorporate a knowledge-based model into the ESN, we feed data generated by these models alongside the primary data being predicted as input to the ESN, as illustrated in Fig. 1(b). Section III A provides a detailed explanation of the knowledge-based models used for each data set.

B. Deep echo state network implementations

The key component of a deep echo state network is the separation of a single reservoir into $m > 1$ sparsely connected hierarchical layers, with the provision that the input layer connects to layer 1, layer m connects to the output layer, and layer i connects to layer $i + 1$ for $1 \leq i < m$. This basic approach, which we refer to as a deep ESN (DESN), is depicted in Fig. 2(a). Figure 2 also illustrates five additional ways to implement a deep ESN using hierarchical designs. The variations among these structures lie in whether the input data are also fed directly into layers $i > 1$ and/or the output layer, and in whether output from layers $i < m$ is passed directly to the output layer.

For the baseline DESN [Fig. 2(a)], where only layer 1 receives the input data and the output layer receives only the states from layer

m , for each layer $k = 2, 3, \dots, m$, the state at time t is computed as follows:

$$h_t^k = (1 - \alpha_k)h_{t-1}^k + \alpha_k \tanh(W_r^k h_{t-1}^{k-1} + W^k h_{t-1}^k), \quad (4)$$

where W_r^k denotes the weight matrix connecting the reservoir in layer $k - 1$ to the reservoir in layer k , W^k is the internal weight matrix of the reservoir in layer k , and α_k is the leaking rate used for layer k .

The output of the DESN at time t is calculated as

$$y_t = f_{\text{out}}(W_{\text{out}}[h_t^m]). \quad (5)$$

When an additional connection is added so that the output layer also receives the input layer [DESN+; see Fig. 2(b)], the reservoir states are computed as described in Eq. (4), and the output at time step t is calculated as follows:

$$y_t = f_{\text{out}}(W_{\text{out}}[x_t; h_t^m]). \quad (6)$$

Figure 2(c) shows the DESN-i structure, in which all reservoirs $k = 2, 3, \dots, m$ receive the input data in addition to the states of the previous layer. The state at time t for each layer $k = 2, 3, \dots, m$ is computed as follows:

$$h_t^k = (1 - \alpha_k)h_{t-1}^k + \alpha_k \tanh(W_r^k h_{t-1}^{k-1} + W^k h_{t-1}^k + W_{\text{in}}^k x_t), \quad (7)$$

and the output is calculated similarly to Eq. (5).

The DESN-o design [see Fig. 2(d)], which along with the DESN-io design below were proposed previously,¹⁹ is similar to that of the basic DESN; however, in this case, the output layer receives the states of all layers, and only layer 1 receives the input. Thus, the states of layers $k = 2, 3, \dots, m$ are computed as described in Eq. (4),

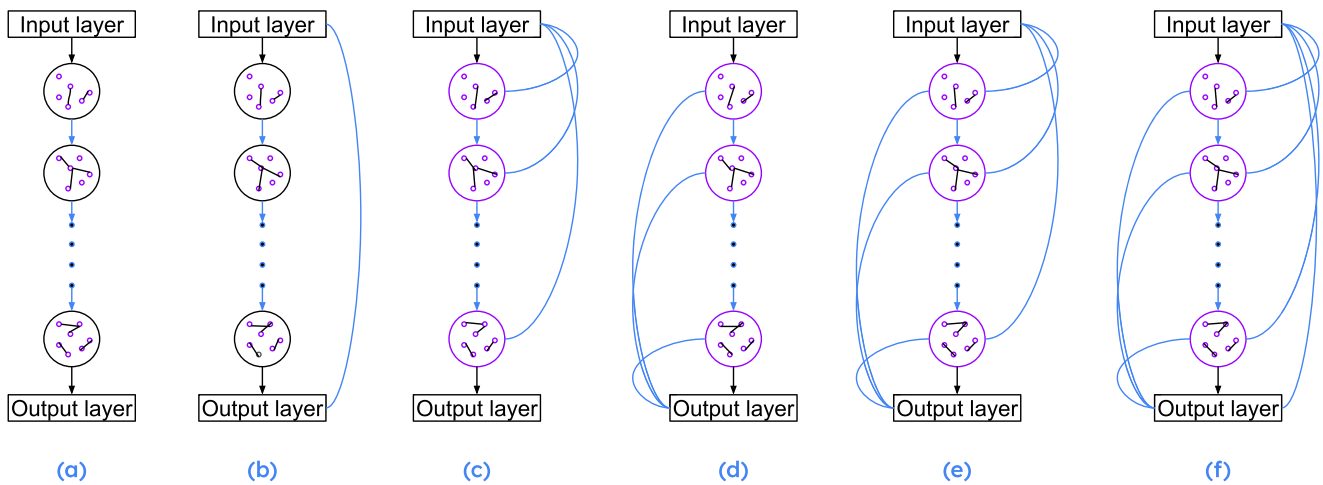


FIG. 2. Deep ESN structures. (a) Basic deep ESN (DESN), (b) deep ESN with a connection from the input layer to the output layer (DESN+), (c) deep ESN with connections from the input layers to the interior layers (DESN-i), (d) deep ESN with connections from the interior layers to the output layer (DESN-o), (e) deep ESN with all interior layers connected to both input and output (DESN-io), and (f) DESN-io with additional connection from the input to the output (DESN-io+).

and the output at each time step is given by

$$y_t = f_{\text{out}}(W_{\text{out}}[h_t^1; h_t^2; \dots; h_t^m]). \quad (8)$$

Combining the DESN-i and DESN-o design features results in the DESN-io variation [Fig. 2(e)], with the states of the reservoirs in each layer at time t for layers $k = 2, 3, \dots, m$ computed using Eq. (7), and the output computed using Eq. (8).

The sixth design, DESN-io+, extends the DESN-io structure by adding a direct connection from the input to the output, as illustrated in Fig. 2(f). Accordingly, the states of the reservoirs at time t for layers $k = 2, 3, \dots, m$ are found using Eq. (7), and the output is computed using the following equation:

$$y_t = f_{\text{out}}(W_{\text{out}}[x_t; h_t^1; h_t^2; \dots; h_t^m]). \quad (9)$$

We also consider hybrid variations of these six designs that incorporate input from a mechanistic model. The DHESN, DHESN+, DHESN-i, DHESN-o, DHESN-io, and DHESN-io+ structures following designs similar to those shown in Fig. 2, with the input including both the data we aim to predict and the knowledge-based data [see Fig. 1(b)].

C. Bayesian optimization

We use Bayesian optimization in several contexts in this work. Bayesian optimization relies on two main components: a probabilistic surrogate model, comprising a prior over the unknown objective function and an observation model, and a loss function, often based on simple or cumulative regret, that evaluates the quality of query sequences. The framework iteratively minimizes expected loss and updates the prior using observed query outcomes to refine the posterior over the objective function.²³

As discussed below in Sec. III, we use Bayesian optimization to identify an appropriate set of parameters for the mathematical knowledge-based model within our hybrid framework for the zebrafish data set, by minimizing the sum of absolute error between the model-generated data and the experimental zebrafish training data. In addition, we use Bayesian optimization for hyperparameter tuning across all the models in our study to find the set of hyperparameters that generate the lowest error. Specifically, by modeling the relationship between machine learning model performance and hyperparameters using Gaussian processes, hyperparameter tuning is formulated as an optimization problem, which is solved using Bayesian optimization.²⁴ While Bayesian optimization has been applied to hyperparameter tuning in various machine learning models,²⁴ to the best of our knowledge, this is the first time it has been used for both flat and deep ESN architectures.

III. IMPLEMENTATION DETAILS

This section provides information on the data sets used for assessing prediction accuracy, including the knowledge-based models considered, as well as details on error assessment and selection of hyperparameter values.

A. Data sets

We evaluated the proposed methods using two data sets: a time series obtained by numerically integrating the Mackey–Glass

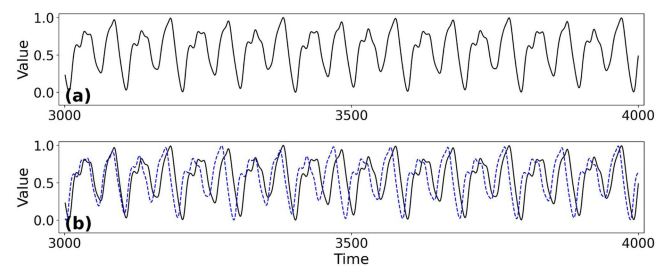


FIG. 3. Mackey–Glass data set; for clarity, only a subset of the 15 000 time-unit-long data set is shown. Mackey–Glass time series alone (a) and with knowledge-based model input [(b), blue].

model and a cardiac voltage data set recorded experimentally from a zebrafish heart. For each data set, the data were split into a training set (80%), with the first 1000 points (corresponding to 1000 time units for the MG data set and the first 1 s of the ZF data set) of that subset reserved for pre-training, and a test set (20%). The data values were normalized using min-max scaling to constrain all values within the range $[0, 1]$.

1. Mackey–Glass data set

The Mackey–Glass (MG) model²¹ describes white blood cell production and is widely used to form complex or chaotic time series. The model is defined by the following delay differential equation with dimensionless time units:

$$\frac{dx}{dt} = \beta \frac{x(t - \tau)}{1 + x^c(t - \tau)} - \gamma x(t), \quad (10)$$

where β , γ , and c are constant parameters set to 0.2, 0.1, and 10, respectively. Moreover, τ is set to 17, as the system exhibits chaotic behavior for $\tau \geq 17$. We generated a time series by solving the MG model using a fourth-order Runge–Kutta method with a time step of 1 time unit for 15 000 time units. The knowledge-based model was generated by adding a small perturbation $\epsilon = 0.1$ to the parameter c . The resulting knowledge-based time series that was used as an additional input in the hybrid approaches is shown in blue in Fig. 3.

2. Experimental cardiac voltage dataset

We also tested the prediction methods on a complex real-world data set of voltage values recorded from one cell in a zebrafish heart (ZF data set). Complexity in the time series was obtained by recording the data under a pacing protocol that was not a constant period; instead the protocol kept constant the time interval between action potentials, whereas the action potentials could vary in duration. In zebrafish hearts, this protocol produced irregular dynamics.¹³ The ZF data set consists of 20 567 recorded voltage values spaced 1 ms apart.

For the knowledge-based model, we tested two mathematical models capable of approximating the voltage dynamics of a cardiac cell, the three-variable Fenton–Karma model (FK)²⁵ and the two-variable Corrado–Niederer (CN) modification²⁶ of the Mitchell–Schaeffer model.²⁷ We also considered a scenario in which

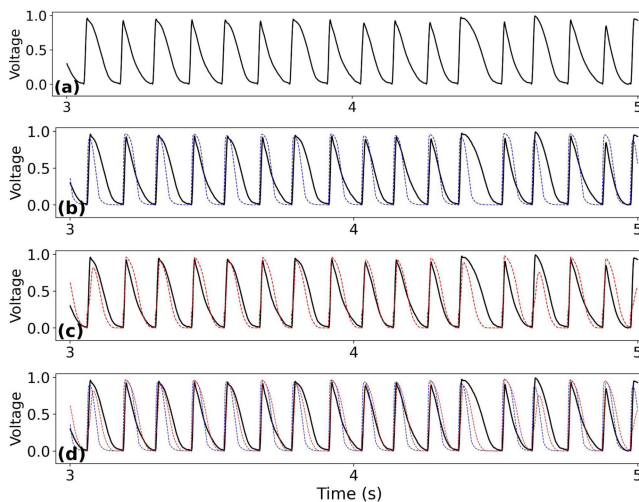


FIG. 4. Experimental cardiac voltage data set; for clarity, only a subset of the 20.6 s-long data set is shown. Voltage time series alone (a) and with knowledge-based model input from the Fenton-Karma model [(b), blue], Corrado-Niederer model [(c), red], and both models (d) superimposed.

both models were used simultaneously. Both models were solved using forward Euler with a time step of 1 ms.

To ensure that the model would produce output that was not extremely inconsistent with the data due to parameter choices, but without fitting the complete model to the data, we used Bayesian optimization to fit just two parameters for each model, resulting in action potentials with similar durations (see Fig. 4). For the fitting process, both τ_r and τ_{si} for the FK model were selected from the interval (25, 35), whereas those for the CN model were chosen from the intervals (0.1, 10) and (1, 20) for τ_{in} and τ_{out} , respectively. The full set of parameter values used for the FK model (based on a fitting²⁸ to the Beeler-Reuter model²⁹) was $\tau_v^+ = 3.33$, $\tau_{v1}^- = 19.6$, $\tau_{v2}^- = 1250$, $\tau_w^+ = 870$, $\tau_w^- = 41.0$, $\tau_d = 0.25$, $\tau_o = 12.5$, $\tau_r = 33.76$, $\tau_{si} = 33.95$, $k = 10.0$, $u_c^{si} = 0.85$, $u_c = 0.13$, and $u_v = 0.04$. For the CN model, the parameter values were set to $\tau_{in} = 0.3711$, $\tau_{out} = 13.74$, $\tau_{open} = 40$, $\tau_{close} = 20$, and $v_{gate} = 0.13$. All time constants for both models are in units of ms.

To elicit action potentials, an external stimulation of magnitude 0.2 was applied for 1 ms. The series of stimulus timings in the form of a binary vector with non-zero entries where stimuli should be applied was included as an additional input to the network along with the data set. The knowledge-based models were stimulated at the same time points as the experimental time series.

B. Hyperparameter selection

We considered reservoir sizes of 96, 144, 240, and 368 neurons for all flat architectures. For the deep architectures, we used the same total number of neurons, distributed across multiple layers beginning with layer 1 as the largest layer with the number of neurons decreasing in deeper layers as follows: two-layer architectures used 64 and 32 neurons; three-layer architectures used 64,

48, and 32 neurons; four-layer architectures used 96, 64, 48, and 32 neurons; and five-layer architectures used 128, 96, 64, 48, and 32 neurons.

The input weight matrix scale for each input was selected from the interval [0.05, 0.2]. The connection probability for intra- and inter-reservoir neurons was chosen from the interval [0.02, 0.1], the spectral radius from [0.8, 1.2], and the leaking rate was allowed to vary over [0.5, 1].

In addition, we allowed different hyperparameter values to be used for each layer, meaning that deeper architectures required tuning a greater number of hyperparameters. To manage this task, we applied Bayesian optimization for 20, 30, 40, 50, and 60 iterations for single-layer, two-layer, three-layer, four-layer, and five-layer networks, respectively.

C. Error assessment

We measured the prediction error using the root mean square error (RMSE), defined as

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (\text{prediction}_t - \text{target}_t)^2},$$

where prediction_t , target_t , and n refer to the prediction for time t , the actual data set value at time t , and the number of points in the test data set, respectively.

To reduce uncertainty in our evaluation, for each scenario considered, we repeated the Bayesian optimization process five times to obtain five networks and reported the average across those five networks as the final error for each case. Specific time-series predictions shown to demonstrate accuracy for a particular scenario were selected as a case close to the average error achieved over the five cases considered for that scenario.

IV. RESULTS

A. Baseline ESN and HESN performance

First, we considered the performance of the baseline flat networks for predicting the time series. Figure 5 shows the error achieved for both the Mackey-Glass (MG) and zebrafish (ZF) data sets for different numbers of neurons (corresponding to the total numbers of neurons in the deep networks that will be considered later). The solid lines indicate results for the basic ESN structure, while the dashed lines show results after adding an additional connection from the input layer to the output layer (ESN+). For the MG data set, there was a trend toward decreasing error with more neurons, but any trend was less clear for the ZF data set. Little difference was seen between the ESN and ESN+ structures for both cases, although the ESN+ structure appears to have a slight improvement in accuracy in most cases. Time-series predictions using the ESN+ structure with 96 and 368 neurons can be seen for the MG and ZF data sets in the top two rows of Figs. 6 and 7, respectively. [See Figs. S1(a) and S2(a) in the supplementary material for corresponding predictions using the ESN structure with 368 neurons.] For the MG data set, a clear improvement in prediction accuracy can be seen with more neurons, with errors being reduced to less than half when the larger network was used. For the ZF data set, the error

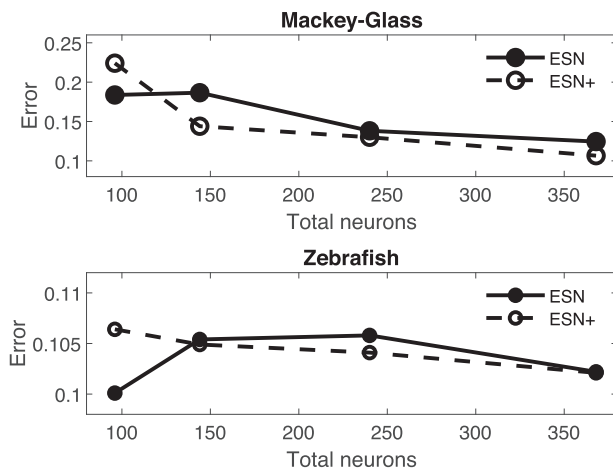


FIG. 5. Error obtained for flat network structures for the Mackey-Glass and zebrafish data sets as a function of the number of neurons. ESN and ESN+ (direct connection from input to output layer) architectures are considered.

decrease with more neurons was slight, and while certain features demonstrated lower error, such as the action potentials between 16.5 and 17.5 s, other features experienced greater error, such as the two action potentials beginning at 20 s.

We also considered whether improvement could be obtained by adding a knowledge-based model (HESN structure). For the MG data set, the hybrid model was the MG model with a modified parameter value, as explained in Sec. III A 1. For the ZF data set, three different hybrid model configurations were tested, including output from the FK model, the CN model, or both as the knowledge-based model(s). In each case, we also considered the effect of including a connection from the input layer to the output layer. As shown in Fig. 8, the MG data set did not show a benefit from the HESN approach over the baseline ESN for any number of neurons tested; error actually increased in all cases with the hybrid approach. In contrast, for the ZF data set, the prediction error decreased relative to the baseline ESN approach for all hybrid options considered, although the error remained mostly the same for different network sizes and regardless of the presence of a direct connection from the input to the output layer. While each choice of knowledge-based model resulted in a decrease in error relative to the baseline ESN, input from the FK model led to the smallest reduction. The error decreases achieved using both models and the CN model alone were quite similar, with error changes within 2% by using both models relative to using only the CN. Thus, selecting the CN model with or without additional input from the FK model most reliably improved on the baseline ESN and ESN+ performance.

The lower rows in Figs. 6 and 7 show time-series predictions using the HESN+ structure with 96 and 368 neurons can be seen for the MG and ZF data sets, respectively [see Figs. S1(b) and S2(b)]

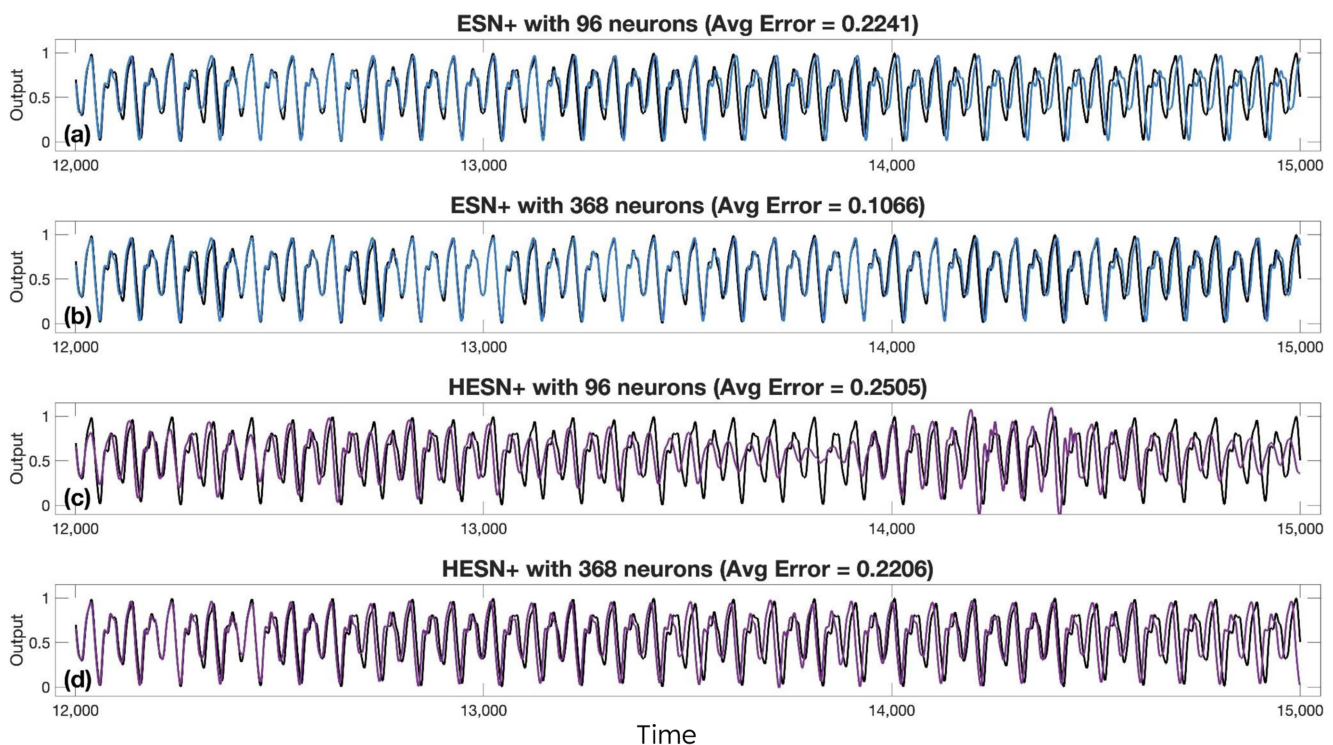


FIG. 6. Time-series predictions for the MG data set using the ESN+ structure with (a) 96 and (b) 368 neurons and the HESN+ structure with (c) 96 and (d) 368 neurons. Data set is shown in black and predictions in color.

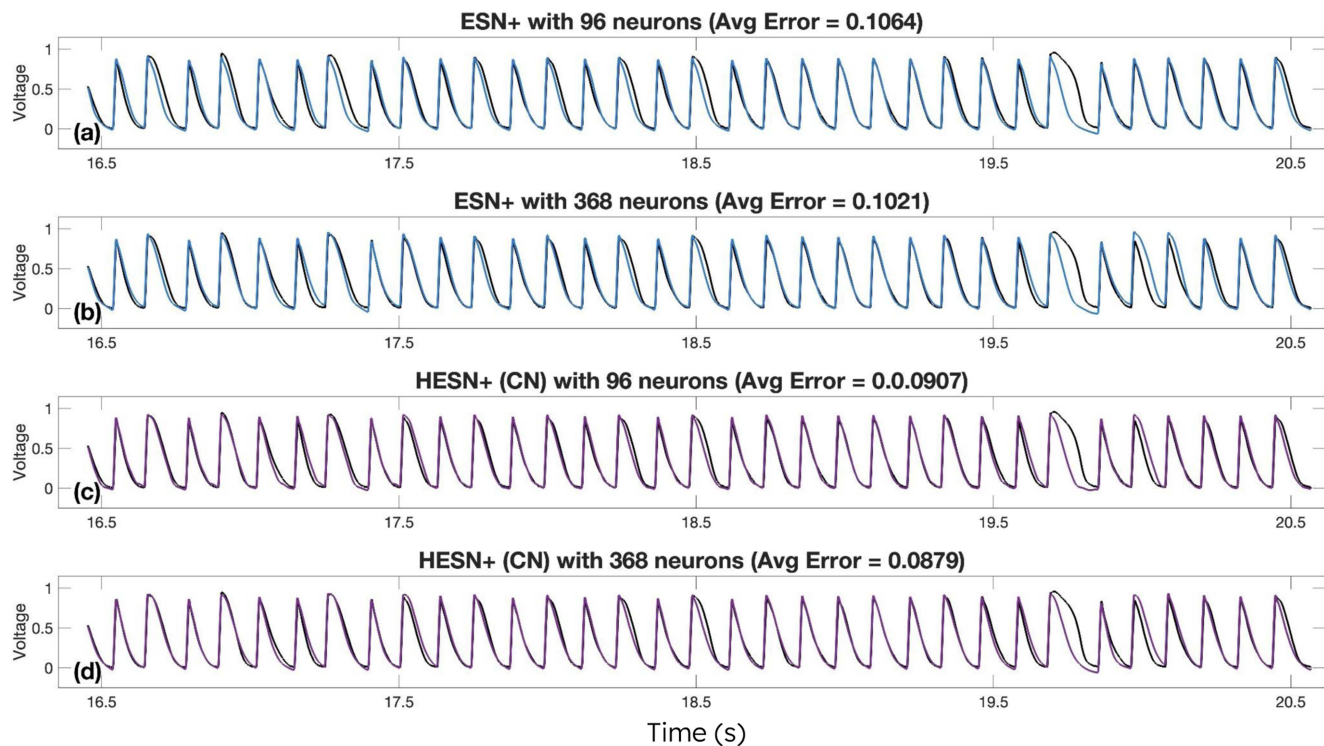


FIG. 7. Time-series predictions for the ZF data set using the ESN+ structure with (a) 96 and (b) 368 neurons and the HESN+ structure with (c) 96 and (d) 368 neurons. Data set is shown in black and predictions in color. For the HESN and HESN+ cases, the CN model was used as the knowledge-based model.

in the [supplementary material](#) for corresponding predictions using the HESN structure with 368 neurons]. As discussed already for the error results (see [Fig. 8](#)), the predictions using the HESN+ approach did not show a benefit over the ESN+ approach, although the HESN+ prediction using 368 neurons improved significantly over the prediction using 96 neurons. For the ZF data set, HESN+ predictions using the CN model demonstrated improvement relative to the ESN+ approach, especially for the first half of the predicted time.

B. Deep ESN and HESN

We aimed to improve on the baseline ESN and HESN predictions by adding structure to the network through organizing the reservoir into layers, as described in [Sec. II B](#). As a first test of a deep structure for the network, we added more layers to the network with decreasing numbers of neurons in each layer, with the restriction that internal layers had no direct connections to either the input or output layers. We compared the results obtained using these structures with those achieved using flat networks with the same total number of neurons. Note that for the deep structures, each increase in the total number of neurons was accompanied by an increase in the number of layers as well, from two layers (corresponding to 96 total neurons) to five layers (corresponding to 368 total neurons).

[Figure 9](#) shows how error scaled with the number of neurons for both the MG and ZF data sets with and without the addition

of input from a knowledge-based model (hybrid approach). As we did for the baseline ESN and HESN cases, we also considered the variations with an additional connection from the input to the output layer. For the MG data set using ESN architectures without a knowledge-based model, the deep architectures showed improvement over the flat counterparts (DESN vs ESN, DESN+ vs ESN+) for smaller numbers of neurons, but not for larger numbers. When the hybrid approach was used, greater accuracy generally was achieved using the deep structure (DHESN vs HESN, DHESN+ vs HESN+), although the error was still higher than that obtained without using the hybrid approach. For the ZF data set, error increased with the use of the deep structure, especially for larger numbers of neurons. Thus, this initial deep ESN approach without connections between the interior layers and input/output layers did not appear to offer a benefit in accuracy for the cases considered, except for the MG data set with small numbers of neurons.

C. Connecting the input layer to the interior layers

We next considered integrating additional connections directly from the input layer to each internal layer (DESN-i and DHESN-i). In this way, the indirect connection from the input layer that becomes filtered through the higher levels in the network was supplemented with the direct connections that might facilitate learning different features of the input signal in different layers. [Figure 10](#)

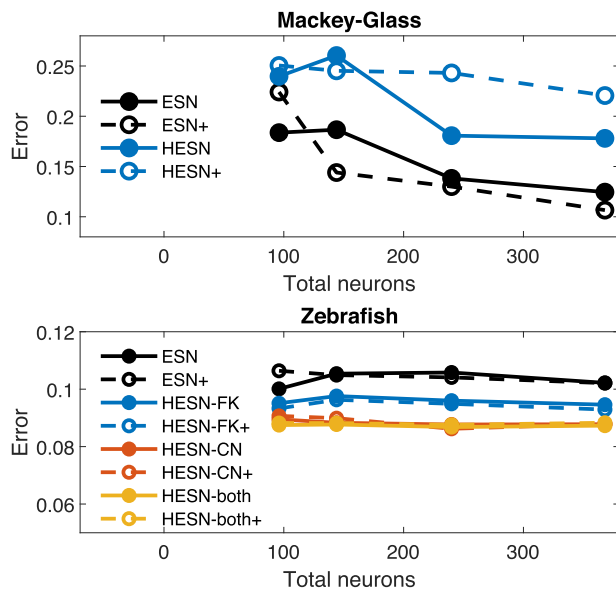


FIG. 8. Error obtained for flat network structures for the Mackey-Glass and zebrafish data sets as a function of the number of neurons when a knowledge-based model was included (HESN). HESN and HESN+ architectures were considered and compared to baseline ESN and ESN+ results. For the zebrafish data set, results are shown for cases with each of two knowledge-based models as well as when both models were used.

shows the results obtained for both the MG and ZF data sets for different network sizes/numbers of layers, compared to the results without these additional connections. Overall, the effect of adding these direct connections from the input layer for both data sets was minimal, especially for small network sizes. For larger network sizes, only modest changes in accuracy resulted from incorporating these additional connections, including both increases (such as DESN-i vs DESN for MG and ZF) and decreases (such as DHESN-i vs DHESN for MG). Thus, for the scenarios we studied, no clear benefit was obtained by modifying the deep network structure to include direct connections from the input layer to each interior layer.

D. Connecting the interior layers to the output layer

We then considered integrating additional connections from each internal layer to the output layer (DESN-o and DHESN-o). In this way, if each layer learned different features, they could be transmitted to the output layer directly. Figure 11 compares the prediction error for different numbers of neurons/numbers of layers with and without these additional connections for both the MG and ZF data sets. With this structure, a decrease in error was seen in every case when compared with the basic deep architecture, with larger improvements for more neurons/more layers. Both the DESN without a knowledge-based model and the DHESN approach benefited. In addition, the error for the DESN-o and DHESN-o decreased in all cases with more layers/more neurons, a behavior that was not observed for either data set with any of the other deep architecture configurations considered so far.

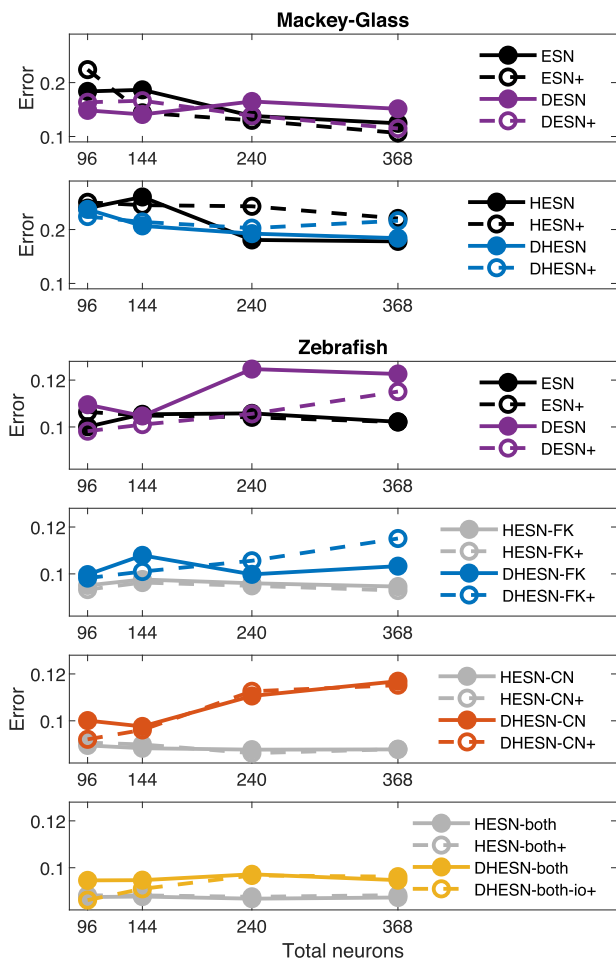


FIG. 9. Error obtained for deep vs flat network structures for the Mackey-Glass and zebrafish data sets as a function of the total number of neurons (96, 144, 240, and 368 neurons correspond to 2, 3, 4, and 5 layers, respectively).

E. Comparisons across all network structures

We compared the results from all architectures discussed so far, as well as from including both connections from the input layer to each internal layer, and from each internal layer to the output layer (DESN-io and DHESN-io), separately considering a direct connection from the input layer to the output layer (DESN-io+ and DHESN-io+). Figure 12 shows the relative change in error for each network structure and size considered, with negative (positive) values indicating a decrease (increase) in the error relative to the baseline flat ESN or HESN. For the MG data set using the ESN approach with no knowledge-based model, no clear trend could be seen until connections were added from the interior layers of deep networks to the output layer, with error clearly decreasing with more layers/more neurons for the DESN-o, DESN-io, and DESN-io+ structures, including an error decrease of up to 65% for the largest network (5 layers, 368 neurons). Little if any benefit was

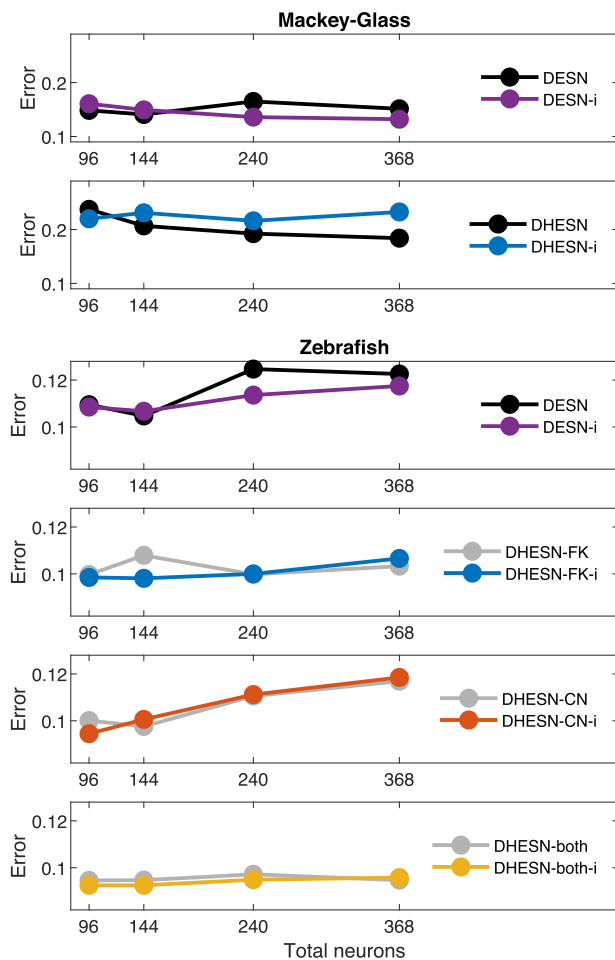


FIG. 10. Error obtained after adding connections from the input layer to each interior layer for the Mackey-Glass and zebrafish data sets as a function of the total number of neurons (96, 144, 240, and 368 neurons correspond to 2, 3, 4, and 5 layers, respectively).

seen when including connections in the deep architectures other than those from the interior layers to the output layers, including from the input layer to the output layer or from the input layer to the interior layers. Figure 13(a) shows the time-series prediction obtained using the DENSE-io+ architecture with 368 neurons, with visible improvement in accuracy compared to the corresponding ESN+ prediction in Fig. 6(b). The corresponding prediction obtained using the DENSE-io structure can be seen in Fig. S3(a), which also improves upon the ESN prediction shown in Fig. S1(a) in the [supplementary material](#).

For the hybrid case, the HESN, DHESN+, and DHESN-i structures showed an increase in the error for deeper networks, but, again, adding connections from the interior layers to the output layers (DHESN-i) led to a decrease in the error, with greater error reductions for deeper networks. Adding additional connections (DHESN-io and DHESN-io+) also led to lower error; although the trend

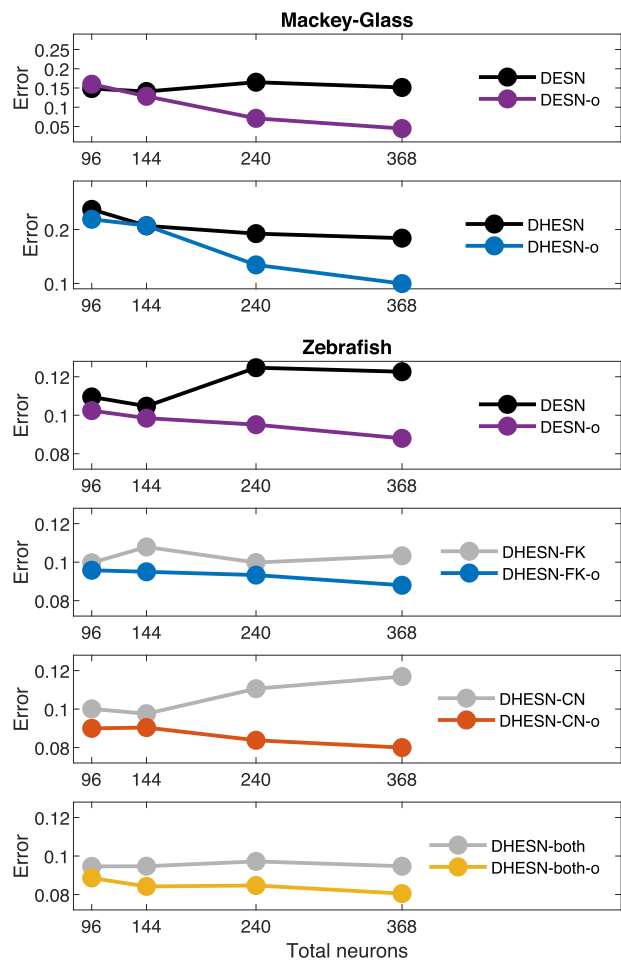


FIG. 11. Error obtained after adding connections from each interior layer to the output layer for the Mackey-Glass and zebrafish data sets as a function of the total number of neurons (96, 144, 240, and 368 neurons correspond to 2, 3, 4, and 5 layers, respectively).

with the number of layers was less clear, for the deepest networks, adding the additional connections from the input layer (DHESN-io+) led to the greatest error reduction of 59%. Although the error decrease relative to the HESN approach was not as large as for the ESN case, the higher error for the flat HESN case (see Fig. 8) led to a greater visible improvement in the time-series prediction when using the DHESN-io+ architecture compared to the HESN+ structure when 368 neurons were used, as shown in Fig. 13(b) relative to Fig. 6(d). Similarly, Fig. S3(b) shows the prediction obtained using the DHESN-io structure, which improves upon the HESN prediction shown in Fig. S1(b) in the [supplementary material](#).

For the ZF data set, Fig. 12 shows the error change relative to the baseline flat ESN or HESN for each network structure considered for the ZF data set across network sizes. For the ESN approach (no knowledge-based model), the deep networks tended to result in increased error unless connections were added from the interior

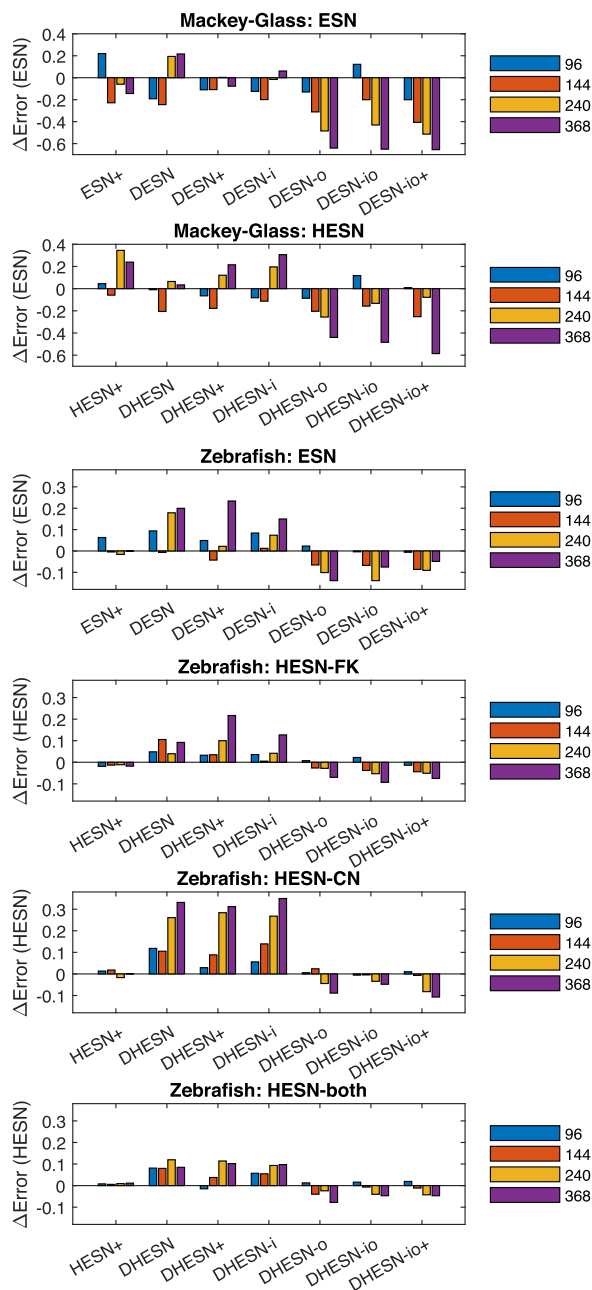


FIG. 12. Change in error for the MG and ZF data sets relative to the corresponding flat ESN or HESN of the same size for different network structures. Network sizes of 96, 144, 240, and 368 neurons correspond to 2, 3, 4, and 5 layers, respectively, in the deep networks.

layers to the output layer (DENSE-o, DENSE-io, and DENSE-io+), similar to what was found for the MG data set, although the magnitude of the error decreases was smaller (the largest decrease was 14%). Similar results were found for the HESN cases considered, with the

highest error seen for the HESN-CN when connections from the interior layers to the output layer were not used and consistent error reductions only observed when those layers were incorporated into the network. Improvements in the time-series predictions when 368 neurons were used can be seen in Fig. 14(a) when using the DENSE-io+ vs the ESN+ architecture [compare Fig. 7(b)] and in Fig. 14(b) when using the DENSE-io+ vs HESN+ architecture [compare Fig. 7(d)]. In particular, the DENSE-io+ network structure achieved better agreement even for the large action potential at about 19.7 s. (See Fig. S4 for corresponding predictions using the DENSE-io and DENSE-io structures.)

We also considered action potential duration (APD) error for the ZF data set. We calculated APD50 values by computing the times when the normalized voltage crossed the value 0.5 during the prediction phase. Figure S5 summarizes key results. In particular, for the shallow ESN+ network, using more neurons led to a decrease in RMSE error in APD of 8% when increasing from 96 to 368 neurons. Incorporating a hybrid component with the CN model as the knowledge-based model led to a greater reduction than adding more neurons, specifically, a 19% reduction when moving to HESN+ with 96 neurons (error also was reduced by 14% for the HESN+ compared to ESN+ with 396 neurons). Finally, the use of deep networks led to significant reductions in APD error compared to the corresponding error in flat networks with the same number of neurons: error decreased by 10% moving from ESN+ to DENSE-io+ and by 33% moving from HESN-CN+ to DENSE-io+.

F. Robustness of predictions

In addition to accuracy benefits from the deep network structures, we also examined the robustness of predictions by analyzing results from 100 randomizations for the MG data set for different network structures. Specifically, we kept the hyperparameter values fixed and simply changed the random seed used to generate different reservoir weights and connections, without using Bayesian optimization to perform any further tuning specific to the network constructed using each random seed. In this way, we could investigate the sensitivity of prediction accuracy to the specific network realization for each of the architectures considered, with consistency across predictions with the desired result.

Figure 15 shows the predictions obtained from the six network structures, including the actual data in black as well as the mean plus and minus the standard deviation in color for each time point. For the baseline ESN+ case, increasing from 96 to 368 neurons resulted in a slightly smaller envelope, indicating an increase in robustness of the predictions with a larger network size. A similar trend was observed for the HESN+ case when increasing from 96 to 368 neurons. When comparing the HESN approach to the baseline ESN approach, it can be seen that the hybrid approach visibly reduced uncertainty in the predictions, especially at earlier times.

For the deep structures, the DENSE-io+ design considerably improved robustness relative to the ESN+ network throughout the prediction window. Addition of the knowledge-based model in the DENSE-io+ structure also resulted in improved robustness of the predicted signal compared to the HESN+ scenario.

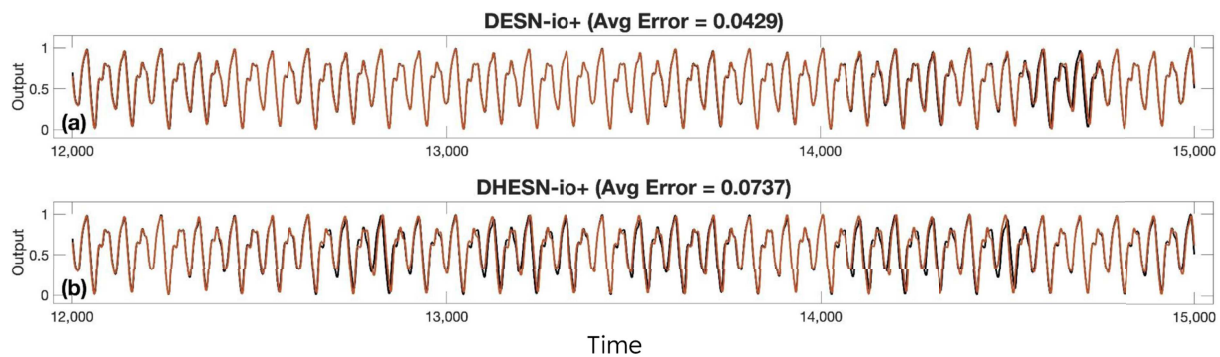


FIG. 13. Time-series predictions for the MG data set using the (a) DESN-io+ and (b) DHESN-io+ structures for 368 neurons (5 layers). Data set is shown in black and predictions in color.

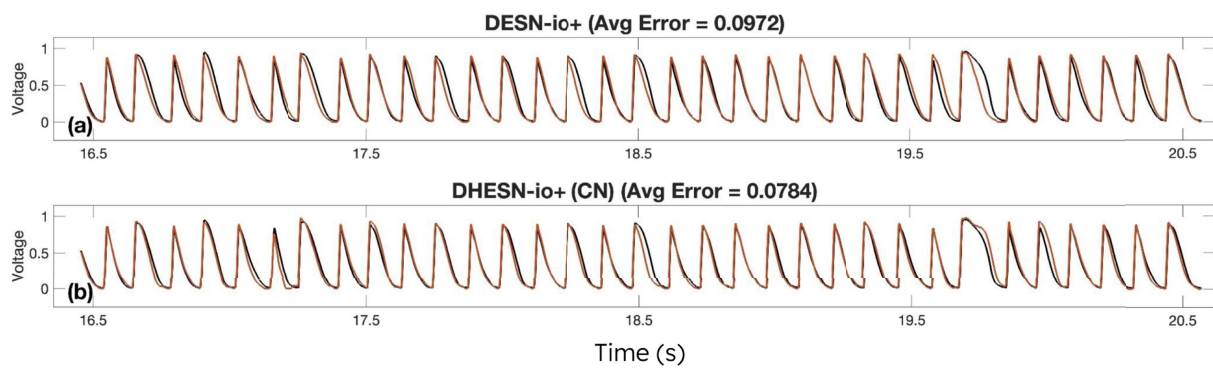


FIG. 14. Time-series predictions for the ZF data set using the (a) DESN-io+ and (b) DHESN-io+ structures for 368 neurons (5 layers). Data set is shown in black and predictions in color. For the DHESN-io+ case, the CN model was used as the knowledge-based model.

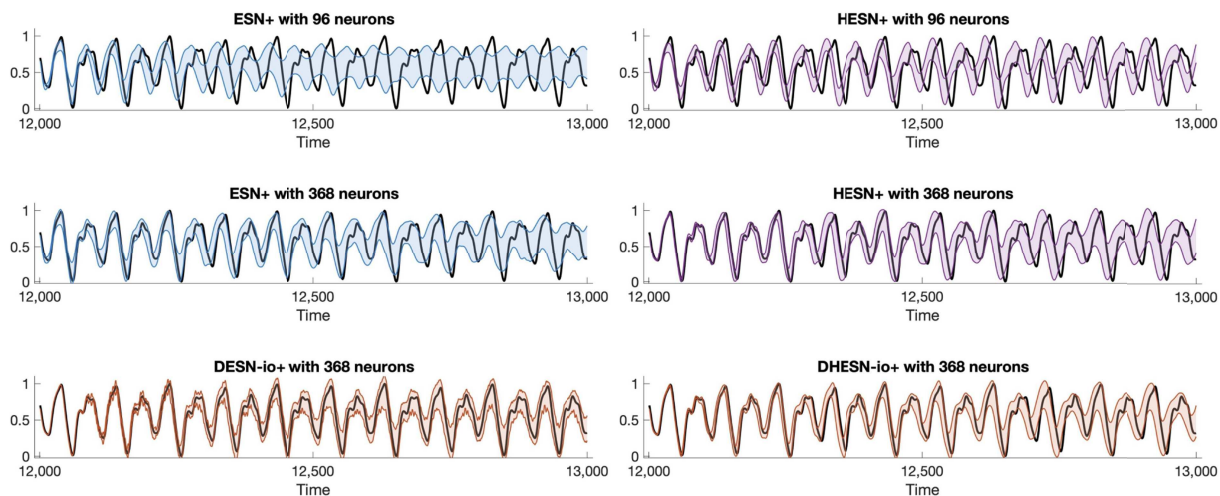


FIG. 15. Robustness of predictions for the MG data set over the first 1000 predicted time units. Black lines represent the actual data, and the colored envelope is determined as mean $\pm$ standard deviation for each time point over 100 randomizations. Randomizations resulting in excessively large error were excluded from the calculations (see text for details).

Moreover, it is important to note that randomization can sometimes result in extremely large errors or even infinity. For this analysis, we excluded cases where the error exceeded 100. Our analysis showed that, for a fixed set of hyperparameters across 100 different randomizations, ESN+ with 96 and 368 neurons produced errors greater than 100 in 2 and 22 cases, respectively, while HESN+ with 96 and 368 neurons did so in 9 and 15 cases. In contrast, both DESN-io+ and DHESN-io+ consistently produced errors below 1 across all 100 randomizations, demonstrating their lower sensitivity to randomization and improved robustness.

V. DISCUSSION

In this paper, we implemented deep ESN approaches for predicting complex time series for two data sets, one derived from the Mackey–Glass model for white blood cell production and one consisting of cardiac voltage recordings from zebrafish hearts. We compared the performance of both single-layer (flat) and deep ESNs with and without a knowledge-based model (HESN and DHESN), and considered the usefulness of adding additional network connections from the input later to the output layer, from the input layer to all deep network layers, and from all deep network layers to the output layer. Overall, we found that deep ESNs could offer advantages in accuracy and robustness over their flat counterparts.

Importantly, we found that deep networks alone were not sufficient to obtain noteworthy accuracy improvements. Error decreases resulted only when direct network connections from the interior layers to the output layer were included. The addition of connections from the input layer to the interior layers or to the output layer did not reliably improve accuracy without connections from each reservoir layer to the output layer. We speculate that the layers learn different temporal dynamics of the data sets that can become obfuscated if they do not directly feed into the output.

We found significant differences in improvement for the two data sets. For the MG data set, decreases in the error of as much as 65% or more were obtained for the largest deep networks (DESN and DHESN) compared to the baseline ESN predictions. For the ZF data set, improvements were more modest, with at most a 14% error reduction compared to the baseline ESN. We expect that one reason for the difference in performance was the higher error obtained for the baseline ESN for the MG data set, which thereby allowed more room for improvement. Another important difference in performance for the two data sets is that the predictions for the ZF time series did not reliably improve with more neurons for flat as well as deep networks, while the MG data set generally benefited from larger networks, suggesting that the 96-neuron networks could capture the complexity of the ZF data set dynamics, but not the MG dynamics.

Incorporation of the knowledge-based model in the HESN and DHESN networks showed additional differences in performance for the two data sets. For flat networks, the MG data set predictions had higher error for the hybrid approach (HESN vs ESN and HESN+ vs ESN+; see Fig. 8). We suspect that the reason for this decrease in accuracy is that the perturbed parameter value used in the knowledge-based model resulted in incorrect input that deviated from the unperturbed data set over time, leading to increasing discrepancies that the ESN could not overcome easily (see Fig. 6). On the other hand, the ZF data set did not have a corresponding

high-fidelity model. Thus, it was easier for the ZF case to benefit selectively from inclusion of a knowledge-based model, for example, to represent sharp features like the action potential upstrokes. Improvement was seen when the hybrid approach was used even in flat networks (see Fig. 8). When deep hybrid ESNs were used with direct connections from the interior layers to the output layer, accuracy improvements occurred in nearly all cases for both the MG and ZF data sets. In particular, the DHESN case produced the lowest error for the troublesome long action potential in the ZF data set [see 19.7 s in Fig. 14(b)]. With regard to the different knowledge-based models considered for the ZF DHESN, for the deepest networks, the FK model was outperformed by the CN model or the combination of the two models, which both achieved similar low error values.

In analyzing the robustness of predictions for different network structures using the MG data set, we found that more neurons led to greater consistency of predictions in flat networks for both the ESN+ and HESN+ network structures, and that the HESN approach achieved less variation. When deep networks were introduced, consistency improved even more throughout the prediction window for both deep ESN and deep hybrid ESN approaches.

We note that deep ESNs require additional design choices compared to basic ESNs. In particular, the number of layers and the distribution of neurons across the layers must be chosen. In this manuscript, we considered a maximum of five layers in our deep ESN architectures. Furthermore, based on preliminary results, we considered network structures obtained by decreasing the number of neurons in deeper layers as the number of layers grew. As an example of an alternative distribution, Fig. 16 shows error obtained for both the MG and ZF data sets using the DESN-io+ structure using the distributions of neurons given in Sec. III B and using an equal distribution of neurons in each layer (e.g., the network with four layers in the “equal” configuration consisted of 60 neurons per layer, for the same total of 240 neurons, rather than 96, 64, 48, and 32 neurons). For the MG data set, the accuracy achieved using the equal configuration was considerably higher than for the standard decreasing configuration. The same finding held for the ZF data set for all network sizes/depths considered, although the difference in error values was much smaller than for the MG data set. This finding suggests that it is important for the deep architectures to include layers that decrease in size.

Compared to earlier deep ESN studies, our approach included several novel aspects. We considered a greater variety of network designs than previous work,¹⁹ with the presence and absence of connections among the input, output, and interior layers, which turned out to have a large effect on the DESN/DHESN performance. To our knowledge, this study also was the first to implement the DHESN approach, and in particular to utilize inputs from two different knowledge-based models, which generally improved prediction accuracy. Our specific implementation of the hybrid approach also differed from that of some earlier implementations that included a direct connection from the knowledge-based model to the output layer.¹¹ Moreover, whereas most previous studies used the same number of neurons for each layer in the DESN structure and compared performance with a single-layer ESN the same size as any one layer in the DESN, we used the same total number of neurons for both deep and flat structures, thereby allowing for a fair comparison and enabling us to differentiate the effect of the

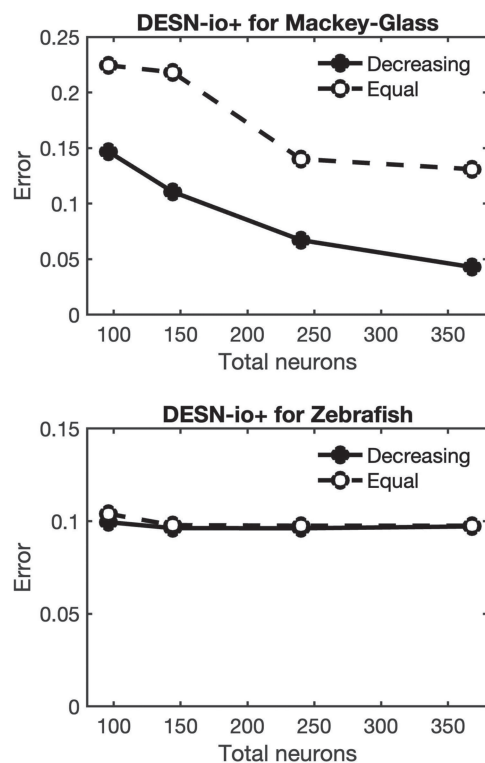


FIG. 16. Error obtained using the DESN-io+ network structure with the standard distribution of neurons across the layers (solid) and with an equal distribution of neurons across the layers (dashed) for the MG and ZF models.

deep architecture from the effect of more neurons. Another novel element was our use of Bayesian optimization for hyperparameter tuning, which has a lower computational cost than options like grid search. We also examined the effects of the random ESN settings for the ESN, HESN, DESN, and DHESN structures considered, leading us to provide some insights into the sensitivity of these approaches to network initialization. Finally, to the best of our knowledge, this study was the first to use deep ESNs for multi-step-ahead prediction.

Our work includes a number of limitations. As noted, the design options for ESNs are quite numerous; we considered only a maximum of five layers and focused primarily on a decreasing scheme for distributing neurons across the layers. Other alternatives could be considered or possibly even optimized. Although we studied some effects of the random components of the network setup (e.g., input weights and reservoir weights as well as reservoir connections), further investigation could be performed. We also considered only a small number of knowledge-based models to use with the ZF data set; it is possible that other model choices could lead to more accurate predictions. Furthermore, we note that different results could be obtained when forecasting different data sets. Finally, the ESN architecture itself imposes limitations in that training is limited to the output layer. Some relaxation of these constraints could be considered to improve deep ESN performance, such as training an intermediate layer along with the output layer, which could

potentially provide some accuracy improvement without increasing the computational cost too much.

Numerous avenues for future work could be pursued to further optimize the deep ESN paradigm for time-series predictions of these and other data sets. Possible directions include examining in more detail the effects of different distributions of neurons across the layers, the use of different knowledge-based models, and the use of multiple knowledge-based model inputs for synthetic data sets like the MG by using multiple perturbed parameter values, as well as implementing other variations of deep ESNs such as deep clustered ESNs.³⁰

SUPPLEMENTARY MATERIAL

Additional figures are available in the [supplementary material](#), as indicated in the text.

ACKNOWLEDGMENTS

This study was supported by NIH Grant No. 2R01HL143450.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Afrouz Delshad: Conceptualization (equal); Investigation (lead); Methodology (equal); Software (equal); Validation (equal); Visualization (equal); Writing – original draft (equal); Writing – review & editing (equal). **Elizabeth M. Cherry:** Conceptualization (equal); Funding acquisition (equal); Investigation (supporting); Methodology (equal); Project administration (equal); Supervision (equal); Validation (equal); Visualization (equal); Writing – original draft (equal); Writing – review & editing (equal).

DATA AVAILABILITY

The data that support the findings of this study are available from the corresponding author upon reasonable request.

REFERENCES

- ¹J. M. Pastore, S. D. Girouard, K. R. Laurita, F. G. Akar, and D. S. Rosenbaum, "Mechanism linking T-wave alternans to the genesis of cardiac fibrillation," *Circulation* **99**, 1385–1394 (1999).
- ²A. Gizzi, E. M. Cherry, R. F. Gilmour, Jr., S. Luther, S. Filippi, and F. H. Fenton, "Effects of pacing site and stimulation history on alternans dynamics and the development of complex spatiotemporal patterns in cardiac tissue," *Front. Cardiac Electrophysiol.* **4**, 71 (2013).
- ³Y. C. Ji, I. Uzelac, N. Otani, S. Luther, R. F. Gilmour, Jr., E. M. Cherry, and F. H. Fenton, "Synchronization as a mechanism for low-energy anti-fibrillation pacing," *Heart Rhythm* **14**, 1254–1262 (2017).
- ⁴S. B. Taieb, G. Bontempi, A. F. Atiya, and A. Sorjamaa, "A review and comparison of strategies for multi-step ahead time series forecasting based on the NN5 forecasting competition," *Expert Syst. Appl.* **39**, 7067–7083 (2012).
- ⁵R. P. Masini, M. C. Medeiros, and E. F. Mendes, "Machine learning advances for time series forecasting," *J. Econ. Surveys* **37**, 76–111 (2023).
- ⁶A. Y. Barrera-Animas, L. O. Oyedele, M. Bilal, T. D. Akinosho, J. M. D. Delgado, and L. A. Akanbi, "Rainfall prediction: A comparative analysis of modern machine

learning algorithms for time-series forecasting," *Machine Learn. Appl.* **7**, 100204 (2022).

⁷I. El Naqa and M. J. Murphy, "What is machine learning?" in *Machine Learning in Radiation Oncology: Theory and Applications* (Springer, 2015), pp. 3–11.

⁸H. Jaeger, "The 'echo state' approach to analysing and training recurrent neural networks—With an erratum note," GMD Technical Report, German National Research Center for Information Technology, Bonn, Germany, Vol. 148, p. 13, 2001.

⁹H. Jaeger, "Adaptive nonlinear system identification with echo state networks," *Adv. Neural Inform. Process. Syst.* **15**, 2056 (2002).

¹⁰M. Lukoševičius and H. Jaeger, "Reservoir computing approaches to recurrent neural network training," *Computer Sci. Rev.* **3**, 127–149 (2009).

¹¹J. Pathak, A. Wikner, R. Fussell, S. Chandra, B. R. Hunt, M. Girvan, and E. Ott, "Hybrid forecasting of chaotic processes: Using machine learning in conjunction with a knowledge-based model," *Chaos* **28**, 041101 (2018).

¹²S. Shahi, F. H. Fenton, and E. M. Cherry, "A machine-learning approach for long-term prediction of experimental cardiac action potential time series using an autoencoder and echo state networks," *Chaos* **32**, 063117 (2022).

¹³S. Shahi, F. H. Fenton, and E. M. Cherry, "Prediction of chaotic time series using recurrent neural networks and reservoir computing techniques: A comparative study," *Machine Learn. Appl.* **8**, 100300 (2022).

¹⁴Y. Tang, Z. Song, Y. Zhu, H. Yuan, M. Hou, J. Ji, C. Tang, and J. Li, "A survey on machine learning models for financial time series forecasting," *Neurocomputing* **512**, 363–380 (2022).

¹⁵S. Elsayed, D. Thyssens, A. Rashed, H. S. Jomaa, and L. Schmidt-Thieme, "Do we really need deep learning models for time series forecasting?" *arXiv arXiv:2101.02118* (2021).

¹⁶P. Lara-Benítez, M. Carranza-García, and J. C. Riquelme, "An experimental review on deep learning architectures for time series forecasting," *arXiv arXiv:2103.12057* (2021).

¹⁷A. Gasparin, S. Lukovic, and C. Alippi, "Deep learning for time series forecasting: The electric load case," *CAAI Trans. Intelligence Technol.* **7**, 1–25 (2022).

¹⁸S. Shastri, K. Singh, S. Kumar, P. Kour, and V. Mansotra, "Time series forecasting of covid-19 using deep learning models: India-USA comparative case study," *Chaos, Solitons Fractals* **140**, 110227 (2020).

¹⁹C. Gallicchio, A. Micheli, and L. Pedrelli, "Deep reservoir computing: A critical experimental analysis," *Neurocomputing* **268**, 87–99 (2017).

²⁰C. Sun, M. Song, D. Cai, B. Zhang, S. Hong, and H. Li, "A systematic review of echo state networks from design to application," *IEEE Trans. Artif. Intelligence* **5**, 23–37 (2022).

²¹M. C. Mackey and L. Glass, "Oscillation and chaos in physiological control systems," *Science* **197**, 287–289 (1977).

²²L. Berezansky and E. Braverman, "Mackey–Glass equation with variable coefficients," *Computers Math. Appl.* **51**, 1–16 (2006).

²³B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. De Freitas, "Taking the human out of the loop: A review of Bayesian optimization," *Proc. IEEE* **104**, 148–175 (2015).

²⁴J. Wu, X.-Y. Chen, H. Zhang, L.-D. Xiong, H. Lei, and S.-H. Deng, "Hyperparameter optimization for machine learning models based on Bayesian optimization," *J. Electron. Sci. Technol.* **17**, 26–40 (2019).

²⁵F. Fenton and A. Karma, "Vortex dynamics in three-dimensional continuous myocardium with fiber rotation: Filament instability and fibrillation," *Chaos* **8**, 20–47 (1998).

²⁶C. Corrado and S. A. Niederer, "A two-variable model robust to pacemaker behaviour for the dynamics of the cardiac action potential," *Math. Biosci.* **281**, 46–54 (2016).

²⁷C. C. Mitchell and D. G. Schaeffer, "A two-current model for the dynamics of cardiac membrane," *Bull. Math. Biol.* **65**, 767–793 (2003).

²⁸F. H. Fenton, E. M. Cherry, H. M. Hastings, and S. J. Evans, "Multiple mechanisms of spiral wave breakup in a model of cardiac electrical activity," *Chaos* **12**, 852–892 (2002).

²⁹G. W. Beeler and H. Reuter, "Reconstruction of the action potential of ventricular myocardial fibres," *J. Physiol.* **268**, 177–210 (1977).

³⁰L. de Oliveira Junior, F. Stelzer, and L. Zhao, "Clustered and deep echo state networks for signal noise reduction," *Machine Learn.* **111**, 2885–2904 (2022).